Synchronous Sequential Circuit

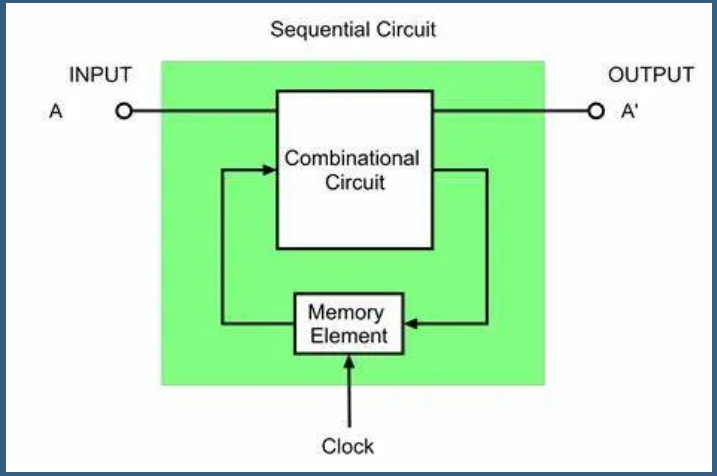
Digital System Fundamentals

Sorayut Glomglome

Learning Outcome

- Draw the state diagram using the Mealy or Moore models.
- Obtain the output from the state diagram or the logic diagram of synchronous sequential circuit.
- Understand the synchronous sequential circuit systhesis procedure.
- Implement / Realize synchronous sequential circuit using the systhesis procedure.
- Realize incompletely specified circuits.

SYNCHRONOUS SEQUENTIAL CIRCUIT



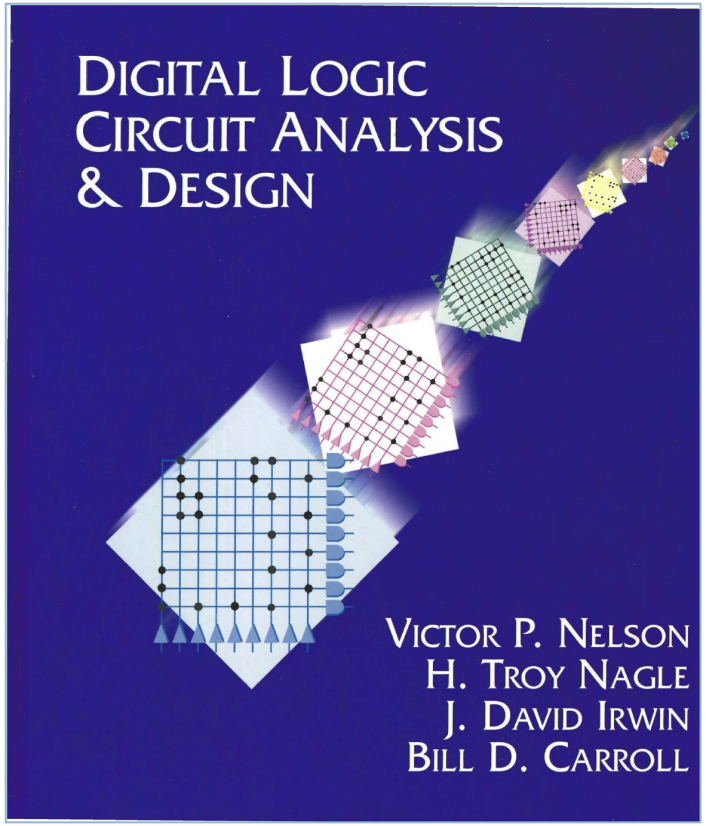
1. Synchronous Sequential Circuit Models

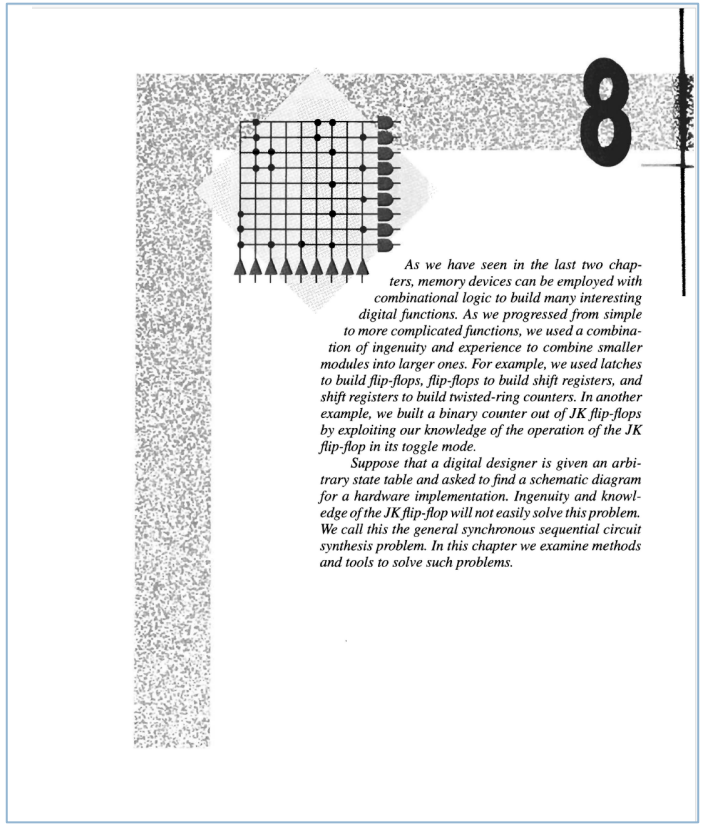
2. Sequential Circuit Analysis

3. Synchronous Sequential Circuit Synthesis

4. Incompletely Specified Circuits

Reference

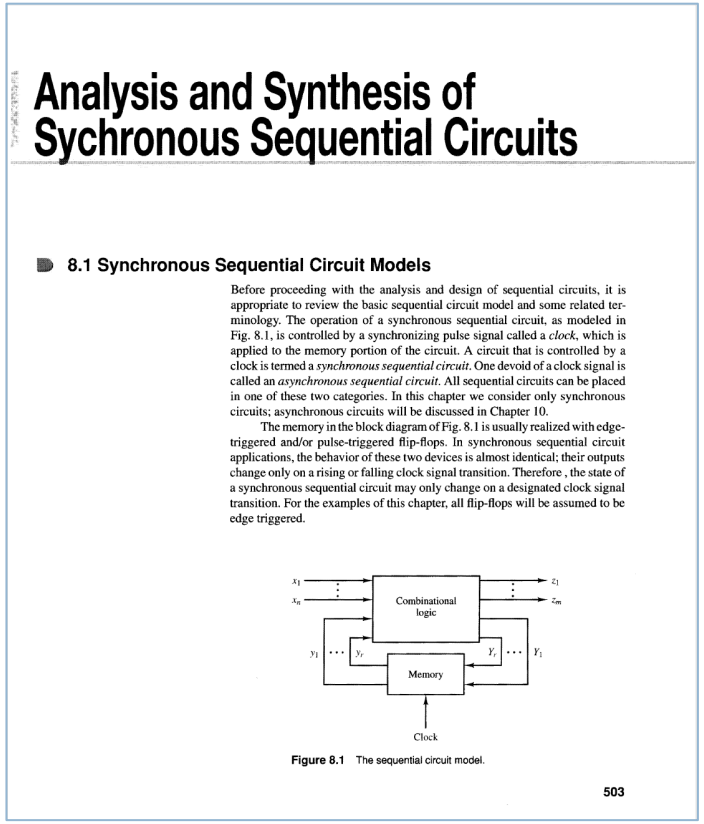




8

As we have seen in the last two chapters, memory devices can be employed with combinational logic to build many interesting digital functions. As we progressed from simple to more complicated functions, we used a combination of ingenuity and experience to combine smaller modules into larger ones. For example, we used latches to build flip-flops, flip-flops to build shift registers, and shift registers to build twisted-ring counters. In another example, we built a binary counter out of JK flip-flops by exploiting our knowledge of the operation of the JK flip-flop in its toggle mode.

Suppose that a digital designer is given an arbitrary state table and asked to find a schematic diagram for a hardware implementation. Ingenuity and knowledge of the JK flip-flop will not easily solve this problem. We call this the general synchronous sequential circuit synthesis problem. In this chapter we examine methods and tools to solve such problems.



Analysis and Synthesis of Sychronous Sequential Circuits

8.1 Synchronous Sequential Circuit Models

Before proceeding with the analysis and design of sequential circuits, it is appropriate to review the basic sequential circuit model and some related terminology. The operation of a synchronous sequential circuit, as modeled in Fig. 8.1, is controlled by a synchronizing pulse signal called a *clock*, which is applied to the memory portion of the circuit. A circuit that is controlled by a clock is termed a *synchronous sequential circuit*. One devoid of a clock signal is called an *asynchronous sequential circuit*. All sequential circuits can be placed in one of these two categories. In this chapter we consider only synchronous circuits; asynchronous circuits will be discussed in Chapter 10.

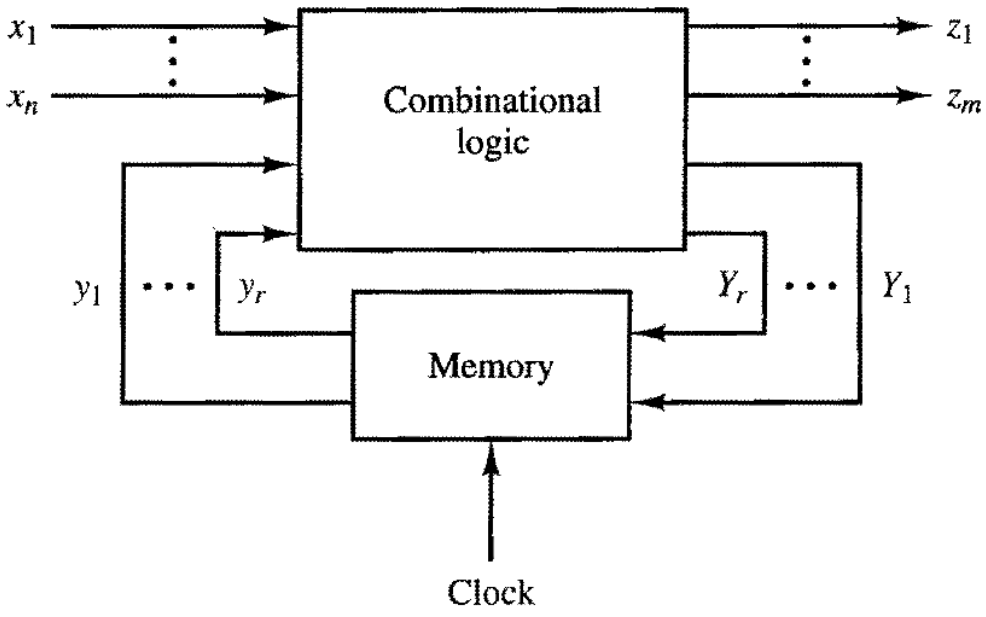
The memory in the block diagram of Fig. 8.1 is usually realized with edge-triggered and/or pulse-triggered flip-flops. In synchronous sequential circuit applications, the behavior of these two devices is almost identical; their outputs change only on a rising or falling clock signal transition. Therefore , the state of a synchronous sequential circuit may only change on a designated clock signal transition. For the examples of this chapter, all flip-flops will be assumed to be edge triggered.

Figure 8.1 The sequential circuit model.

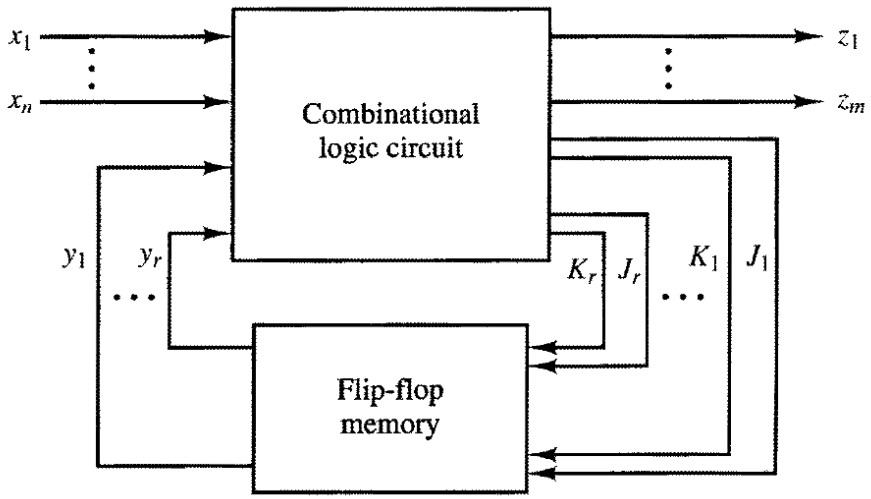
503

1. Synchronous Sequential Circuit Models

Types of Sequential Circuits



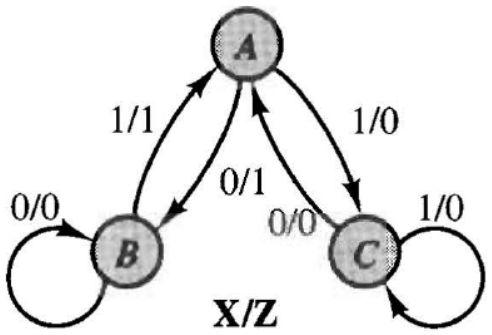
Synchronous sequential circuit



Asynchronous sequential circuit

Mealy Model

- Outputs are functions of both the *inputs* and the *present state*.
- Called a *transition-assigned circuit*
  - because the circuit output is associated with the *state transitions*
  - the arcs in the state diagram



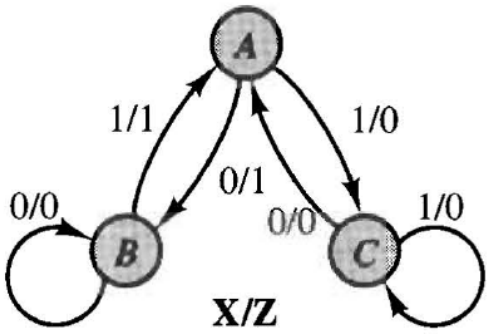
State diagram

Present state	Input x 0	1
A	$B/1$	$C/0$
B	$B/0$	$A/1$
C	$A/0$	$C/0$

Next state/output

State table

Mealy Model Example



Let us determine the output response of the sequential circuit defined in Fig. 8.2 to the input sequence $x = 011010$.

Let us assume that the circuit is initially in state A. At time 0 an input of $x = 0$ is applied. From either the state diagram or the state table, we can read that the output is $z = 1$ and the next state will be B. Continuing for the remaining values in the input sequence, the circuit will behave as follows:

Time:	0	1	2	3	4	5	
Present state:	A	B	A	C	A	C	A
Input:	0	1	1	0	1	0	
Output:	1	1	0	0	0	0	
Next state:	B	A	C	A	C	A	

Hence, this input sequence applied to the machine in state A causes the output sequence

$$z = 110000$$

and leaves the circuit in final state A.

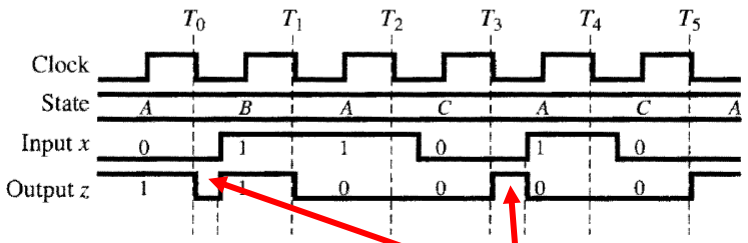


Figure 8.3 Mealy model timing diagram.

Avoid sample unexpected outputs aka glitches

Start with *State B*

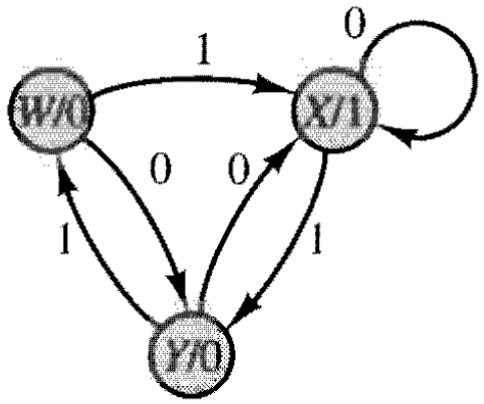
Time:	0	1	2	3	4	5
Present state:						
Input:	0	1	1	0	1	0
Output:						
Next state:						

Start with *State C*

Time:	0	1	2	3	4	5
Present state:						
Input:	0	1	1	0	1	0
Output:						
Next state:						

Moore Model

- Outputs are solely function with the *present state*.
  - Output is inside the circles.



State diagram

Present state	Input x 0	Input x 1	Outputs
W	Y	X	0
X	X	Y	1
Y	X	W	0

State table

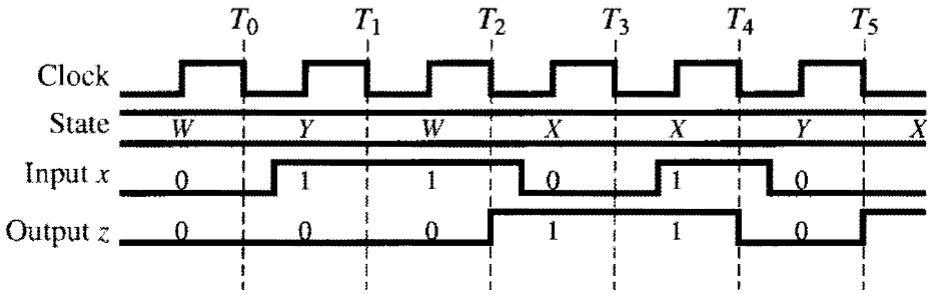
Moore Model

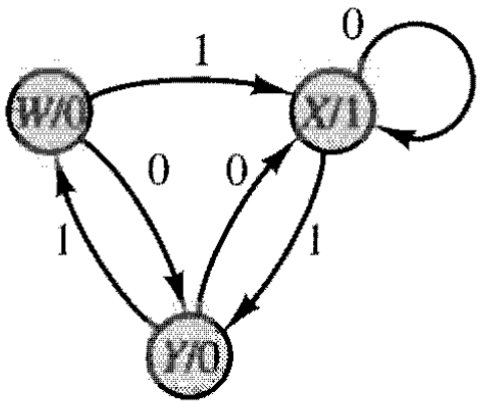
Consider the following input sequence to the Moore model of Fig. 8.4 with the starting state *W*:

Time:	0	1	2	3	4	5	
Present state:	*W*	*Y*	*W*	*X*	*X*	*Y*	*X*
Input:	0	1	1	0	1	0	
Output:	0	0	0	1	1	0	
Next State:	*Y*	*W*	*X*	*X*	*Y*	*X*	

The output is always identified by the present state from either the state diagram or the state table.

Figure 8.5 illustrates the actual circuit timing for the given input sequence. As in the previous example, all state changes occur on the high-to-low transition of the clock.





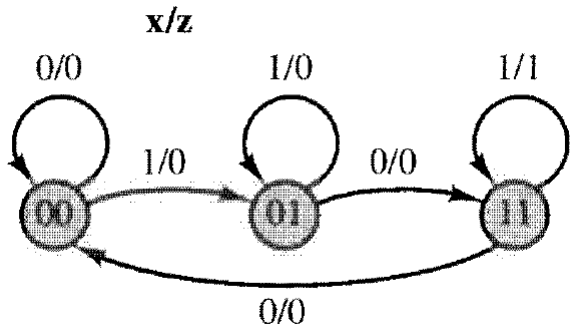
Moore model circuit typically exhibit better behavior that those of a Mealy model circuit; that is, input changes ***do not result in unwanted glitches*** in the outputs.

The primary advantage of using a ***Mealy model*** is that the designer has more flexibility in designing output and state transition functions, and thus ***fewer states*** will be needed than in an equivalent Moore model circuit.

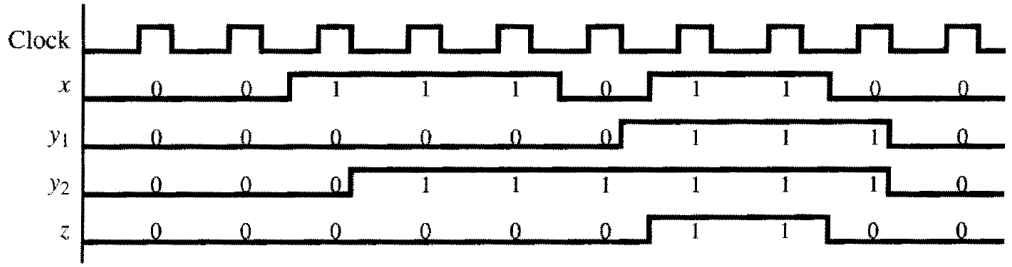
2. Sequential Circuit Analysis

What is the sequential circuit analysis ?

- Analysis is the process of determining the *output response* of a given circuit or circuit model to a given input sequence.

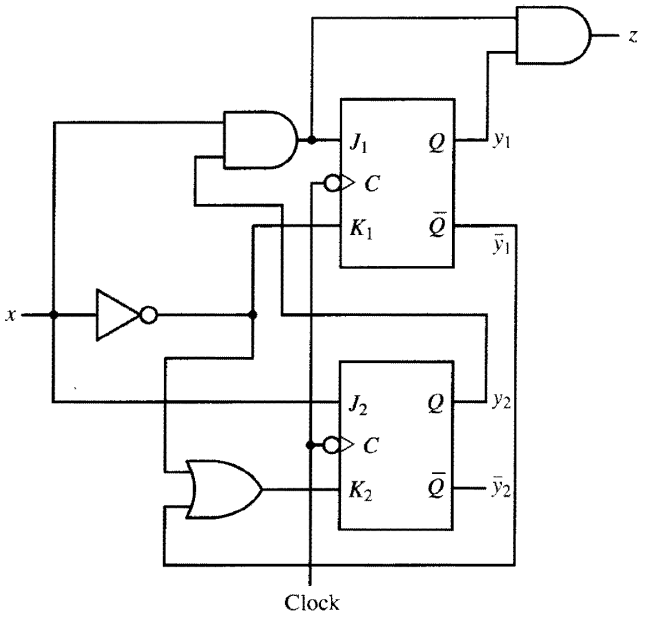


Circuit Analysis

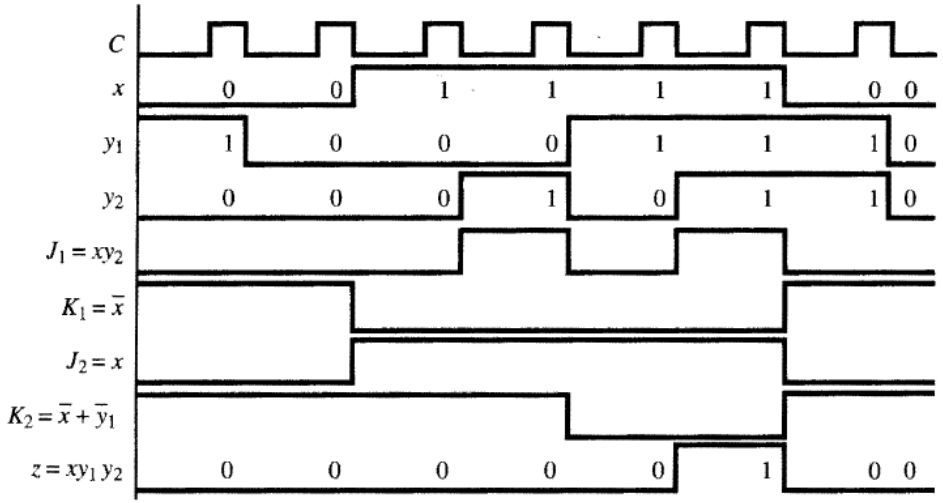


input sequence 001110110

Analysis of Sequential Circuit from Logic Diagrams



$$J_1 = xy_2,$$
$$K_1 = \bar{x},$$
$$J_2 = x,$$
$$K_2 = \bar{x} + \bar{y}_1,$$
$$z = xy_1y_2$$
$$x = 0011110$$
$$y_1^0 y_2^0 = 10$$



step	x	J1	K1	y1	J2	K2	y2	z
1	0	0	1	1	0	1	0	0
2	0	0	1	0	0	1	0	0
3	1	0	0	0	1	1	0	0
4	1	1	0	0	1	1	1	0
5	1	0	0	1	1	0	0	0
6	1	1	0	1	1	0	1	1
7	0	0	1	1	0	1	1	0
				0			0	

$y_1 y_2$ \ x	0	1
00	00/0	01/0
01	00/0	10/0
11	00/0	11/1
10	00/0	11/0

State table

Analysis Procedure

Synchronous Sequential Circuit

Analysis Procedure

Step 1. If a state table or state diagram is given, proceed to steps 6 or 7, respectively. Otherwise, continue. Use combinational logic analysis techniques to determine the flip-flop input equations and the circuit output equations. Go to step 7 if only a timing diagram is needed.

Step 2. Construct K-maps for all the logic equations from step 1.

Step 3. Combine the K-maps for all flip-flop input equations into a single map.

Step 4. Using the characteristic equations of the flip-flops, construct a next-state map.

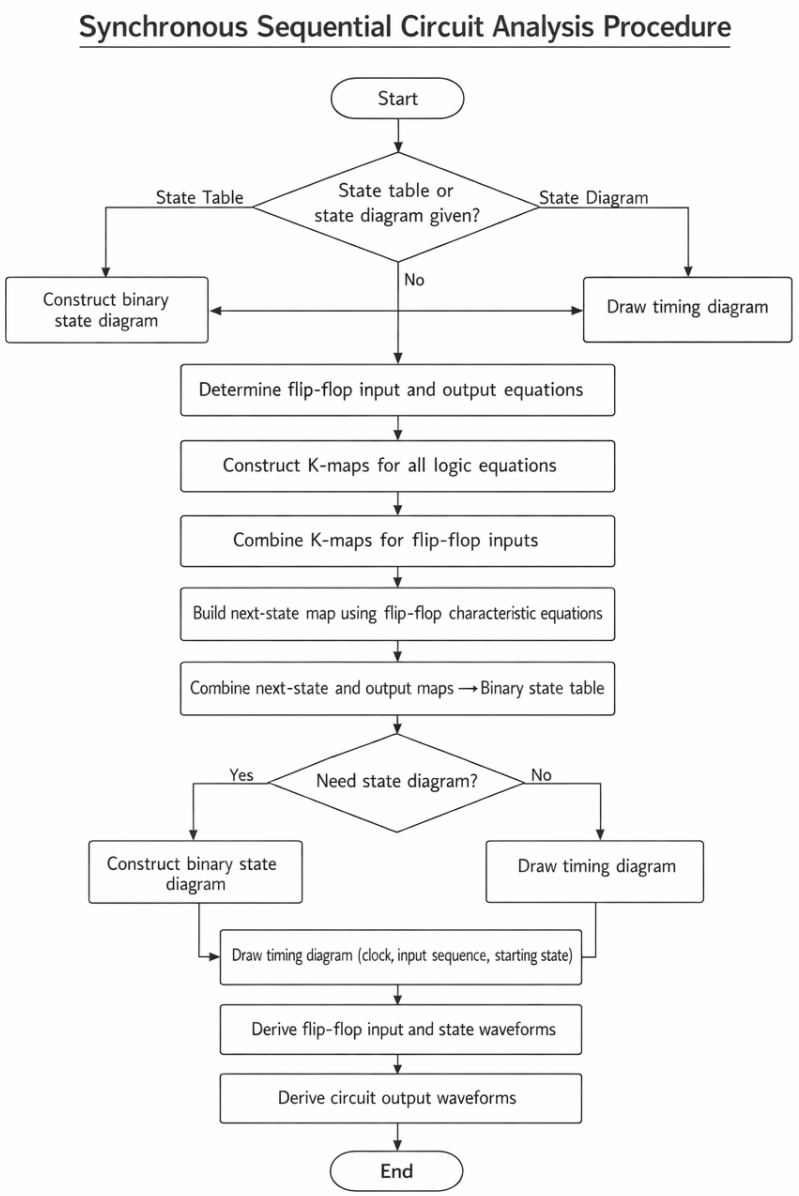
Step 5. Combine the next-state map and the output maps into a single map. This step results in a binary state table.

Step 6. Construct a binary state diagram from the binary state table if desired. Otherwise, go to step 7.

Step 7. Draw a timing diagram showing the clock, the given input sequence, and the starting state.

Step 8. On the timing diagram, derive the waveforms for the flip-flop input(s) and flip-flop state(s) for all the remaining circuit input values.

Step 9. On the timing diagram, derive the waveforms of the circuit output(s).



3. Synchronous Sequential Circuit Synthesis

Design of Synchronous Counters

Step 1: State Diagram

Step 2: Next-State Table

Step 3: Flip-Flop Transition Table

Step 4: Karnaugh Maps

Step 5: Logic Expressions for Flip-Flop Inputs

Step 6: Counter Implementation

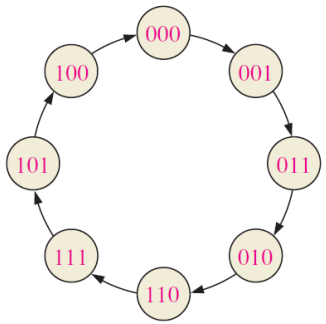


TABLE 9–8

Next-state table for 3-bit Gray code counter.

Present State			Next State		
Q_2	Q_1	Q_0	Q_2	Q_1	Q_0
0	0	0	0	0	1
0	0	1	0	1	1
0	1	1	0	1	0
0	1	0	1	1	0
1	1	0	1	1	1
1	1	1	1	0	1
1	0	1	1	0	0
1	0	0	0	0	0

TABLE 9–9

Transition table for a J-K flip-flop.

Output Transitions			Flip-Flop Inputs	
Q_N		Q_{N+1}	J	K
0	⟶	0	0	X
0	⟶	1	1	X
1	⟶	0	X	1
1	⟶	1	X	0

Q_N: present state
Q_{N+1}: next state
X: "don't care"

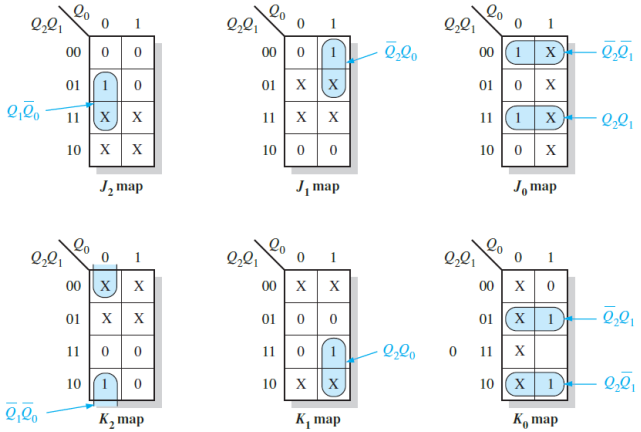
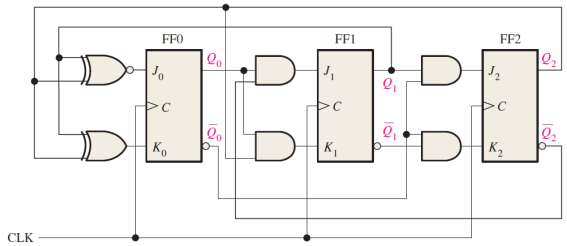


FIGURE 9–28 Karnaugh maps for present-state J and K inputs.

$$J_0 = Q_2Q_1 + \overline{Q}_2\overline{Q}_1 = \overline{Q_2 \oplus Q_1}$$
$$K_0 = Q_2\overline{Q}_1 + \overline{Q}_2Q_1 = Q_2 \oplus Q_1$$
$$J_1 = \overline{Q}_2Q_0$$
$$K_1 = Q_2Q_0$$
$$J_2 = Q_1\overline{Q}_0$$
$$K_2 = \overline{Q}_1\overline{Q}_0$$



Synthesis Procedure

Step 1. From a word description of the problem, derive a state table.

Step 2. Use state reduction techniques to find the minimum state table.

Step 3. Choose a state assignment and generate the state and output transition tables.

Step 4. Determine the memory device or flip-flop to be used and find the flip-flop excitation maps (transition tables).

Step 5. From the excitation maps, produce the logic equations. Also, form output maps and determine the output logic equations.

Step 6. Draw the logic diagram of the sequential circuit using logic equations and the chosen memory devices.

Flip-flop Transition Tables

State transitions $Q(t)$	$Q(t+\varepsilon)$	Required inputs $D(t)$
0	0	0
0	1	1
1	0	0
1	1	1

(a)

State transitions $Q(t)$	$Q(t+\varepsilon)$	Required inputs $S(t)$	$R(t)$
0	0	0	d
0	1	1	0
1	0	0	1
1	1	d	0

(b)

State transitions $Q(t)$	$Q(t+\varepsilon)$	Required inputs $T(t)$
0	0	0
0	1	1
1	0	1
1	1	0

(c)

State transitions $Q(t)$	$Q(t+\varepsilon)$	Required inputs $J(t)$	$K(t)$
0	0	0	d
0	1	1	d
1	0	d	1
1	1	d	0

(d)

Figure 8.22 Flip-flop input tables. **(a)** D flip-flop. **(b)** Clocked SR flip-flop. **(c)** Clocked T flip-flop. **(d)** Clocked JK flip-flop.

TABLE 9–9

Transition table for a J-K flip-flop.

Output Transitions Q_N		Q_{N+1}	Flip-Flop Inputs J	K
0	$\longrightarrow$	0	0	X
0	$\longrightarrow$	1	1	X
1	$\longrightarrow$	0	X	1
1	$\longrightarrow$	1	X	0

Q_N: present state
Q_{N+1}: next state
X: "don't care"

Design Example 1

Let us find a clocked D flip-flop realization for the sequential circuit defined in the state table.

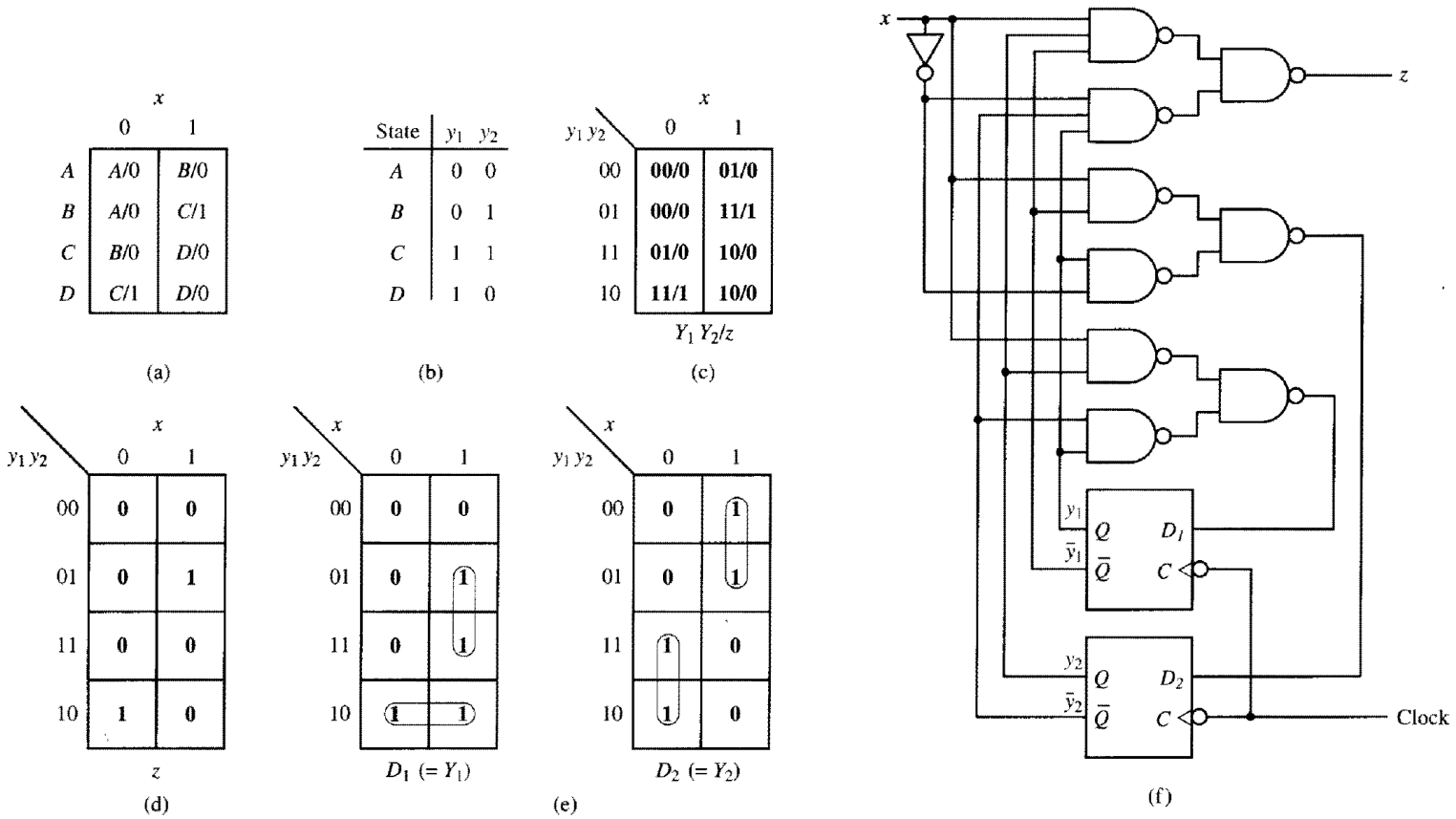


Figure 8.21 An introductory example. **(a)** State table. **(b)** State assignment. **(c)** Transition table. **(d)** Output K-map. **(e)** Excitation K-maps. **(f)** Logic diagram.

State transitions $Q(t)$	$Q(t+\varepsilon)$	Required inputs $D(t)$
0	0	0
0	1	1
1	0	0
1	1	1

Design Example 1 Using JK-FF

y_1y_2 \ x	0	1
00	00/0	01/0
01	00/0	11/1
11	01/0	10/0
10	11/1	10/0

$Y_1\,Y_2/z$

(a)

y_1y_2 \ x	0	1
00	0d	0d
01	0d	1d
11	d1	d0
10	d0	d0

J_1K_1

y_1y_2 \ x	0	1
00	0d	1d
01	d1	d0
11	d0	d1
10	1d	0d

J_2K_2

(b)

$$J_1 = xy_2 \qquad K_1 = \bar{x}y_2$$
$$J_2 = x \oplus y_1 \qquad K_2 = x \odot y_1 = \bar{J}_2$$

y_1y_2 \ x	0	1
00	0	0
01	0	1
11	d	d
10	d	d

J_1

y_1y_2 \ x	0	1
00	d	d
01	d	d
11	1	0
10	0	0

K_1

y_1y_2 \ x	0	1
00	0	1
01	d	d
11	d	d
10	1	0

J_2

y_1y_2 \ x	0	1
00	d	d
01	1	0
11	0	1
10	d	d

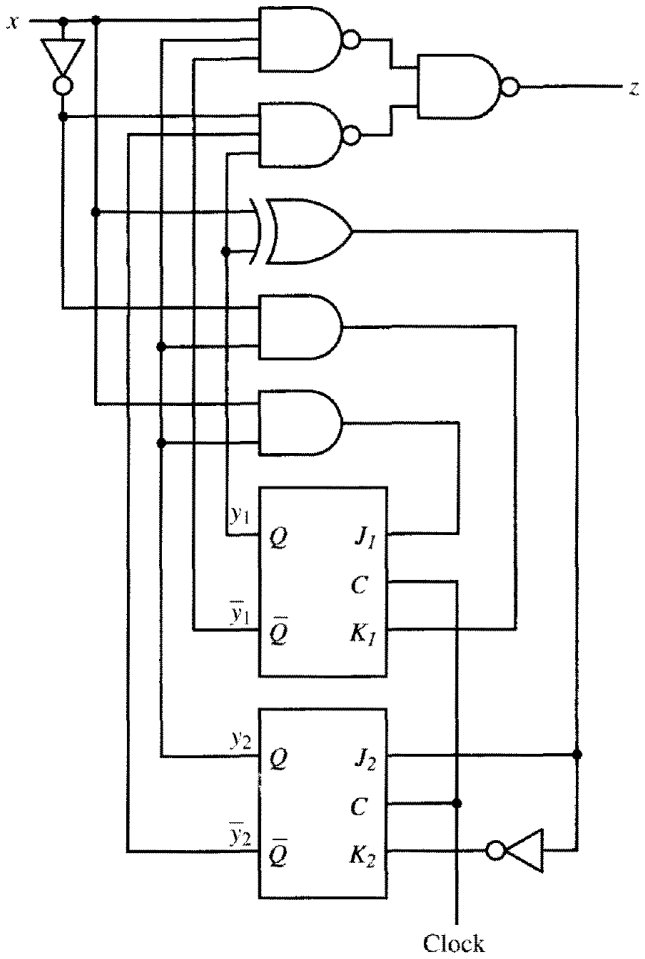
K_2

(c)

State transitions $Q(t)$	$Q(t+\varepsilon)$	Required inputs $J(t)$	$K(t)$
0	0	0	d
0	1	1	d
1	0	d	1
1	1	d	0

Figure 8.23 Generating the excitation maps. **(a)** Transition table. **(b)** Excitation tables. **(c)** Excitation maps.

Design Example 1 Using JK-FF | Logic Diagram



$$J_1 = xy_2 \qquad K_1 = \bar{x}y_2$$
$$J_2 = x \oplus y_1 \qquad K_2 = x \odot y_1 = \bar{J}_2$$

Design Example 2 | Sequence Recognizer 1

Design a synchronous sequential circuit with one input x and one output z that recognizes the input sequence 01.

x = 0101000001111 01

z = 0101000001000 01

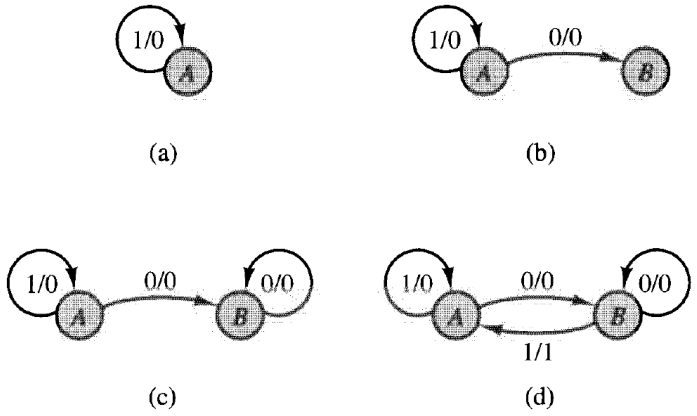


Figure 8.26 Sequence 01 recognizer state diagram.

Design Example 2 | Sequence Recognizer 1

x = 01010000011110 1

z = 01010000010000 1

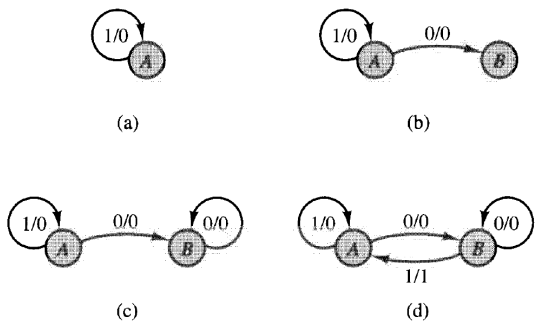


Figure 8.26 Sequence 01 recognizer state diagram.

State transitions $Q(t)$	$Q(t+\varepsilon)$	Required inputs $S(t)$	$R(t)$
0	0	0	d
0	1	1	0
1	0	0	1
1	1	d	0

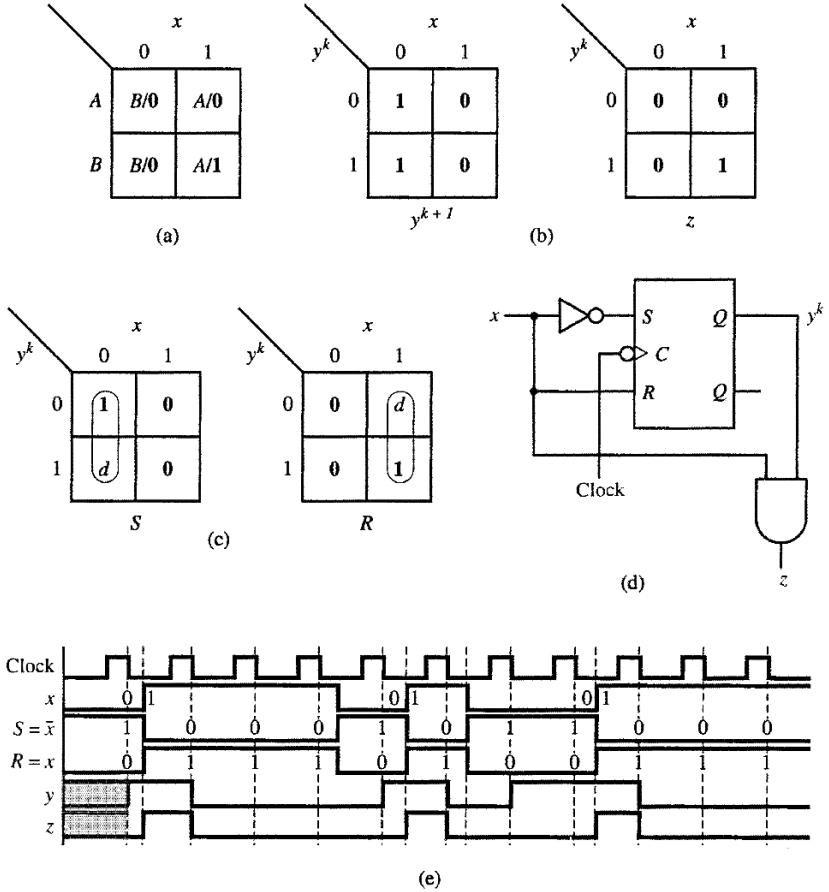


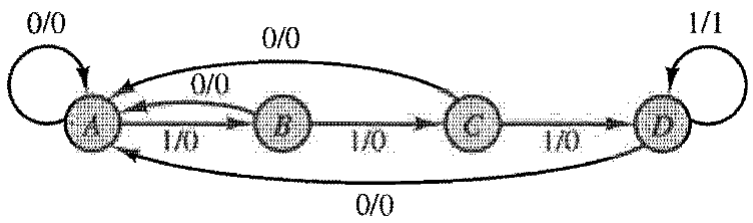
Figure 8.27 Synthesis of a sequence 01 recognizer. **(a)** State table. **(b)** Transition table and output map. **(c)** Excitation maps. **(d)** Logic diagram. **(e)** Timing diagram.

Design Example 3 | Sequence Recognizer 2

Let us design a synchronous sequential circuit with one input line and one output line that recognizes the input string x 1111. The circuit is also required to recognize overlapping sequences, as can be seen in the output string z that results from the following input string x :

$A = 00$
$B = 01$
$C = 10$
$D = 11$

Step 3: State Assignment



Step 1: State Diagram

	x = 0	x = 1
A	$A/0$	$B/0$
B	$A/0$	$C/0$
D	$A/0$	$D/1$
C	$A/0$	$D/0$

Step 2: State Table

$y_1^k y_2^k$	x = 0	x = 1
00	00	01
01	00	10
11	00	11
10	00	11

$y_1^{k+1} y_2^{k+1}$

Step 3: Next State Table

$y_1^k y_2^k$	x = 0	x = 1
00	0	0
01	0	0
11	0	1
10	0	0

z

Step 5: Output Table

Design Example 3 | Output Logic Equation

$y_1^k y_2^k$ \ x	0	1
00	0	0
01	0	0
11	0	1
10	0	0

z

$$z = x y_1 y_2$$

Step 5: Output Table

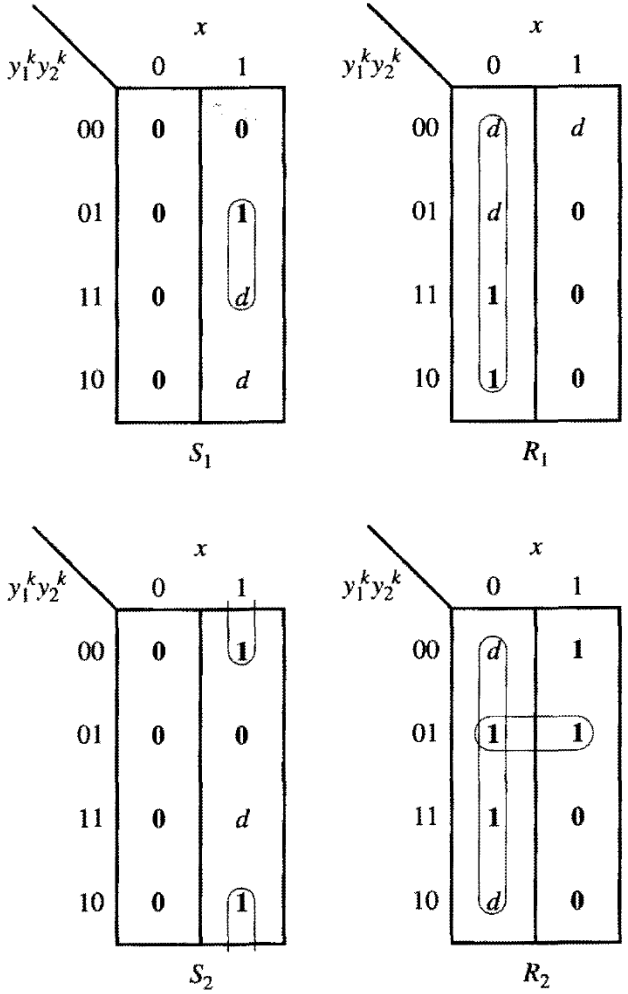
Design Example 3 | SR-FF

$y_1^k y_2^k$ \ x	0	1
00	00	01
01	00	10
11	00	11
10	00	11

$y_1^{k+1} y_2^{k+1}$

State transitions $Q(t)$	$Q(t+\varepsilon)$	Required inputs $S(t)$	$R(t)$
0	0	0	d
0	1	1	0
1	0	0	1
1	1	d	0

$$S_1 = y_2 x, \quad S_2 = \bar{y}_2 x,$$
$$R_1 = \bar{x}, \quad R_2 = \bar{x} + \bar{y}_1 y_2$$



Step 4: Flip-flop transition tables aka excitation maps

Design Example 3 | JK-FF

$y_1^k y_2^k$ \ x	0	1
00	00	01
01	00	10
11	00	11
10	00	11

$y_1^{k+1} y_2^{k+1}$

State transitions $Q(t)$	$Q(t+\varepsilon)$	Required inputs $J(t)$	$K(t)$
0	0	0	d
0	1	1	d
1	0	d	1
1	1	d	0

$$J_1 = y_2 x, \quad J_2 = x, \quad z = x y_1 y_2$$
$$K_1 = \bar{x}, \quad K_2 = \bar{y}_1 + \bar{x}$$

$y_1^k y_2^k$ \ x	0	1
00	0	0
01	0	1
11	d	d
10	d	d

J_1

$y_1^k y_2^k$ \ x	0	1
00	d	d
01	d	d
11	1	0
10	1	0

K_1

$y_1^k y_2^k$ \ x	0	1
00	0	1
01	d	d
11	d	d
10	0	1

J_2

$y_1^k y_2^k$ \ x	0	1
00	d	d
01	1	1
11	1	0
10	d	d

K_2

Step 4: Flip-flop transition maps aka excitation maps

Serial Adder

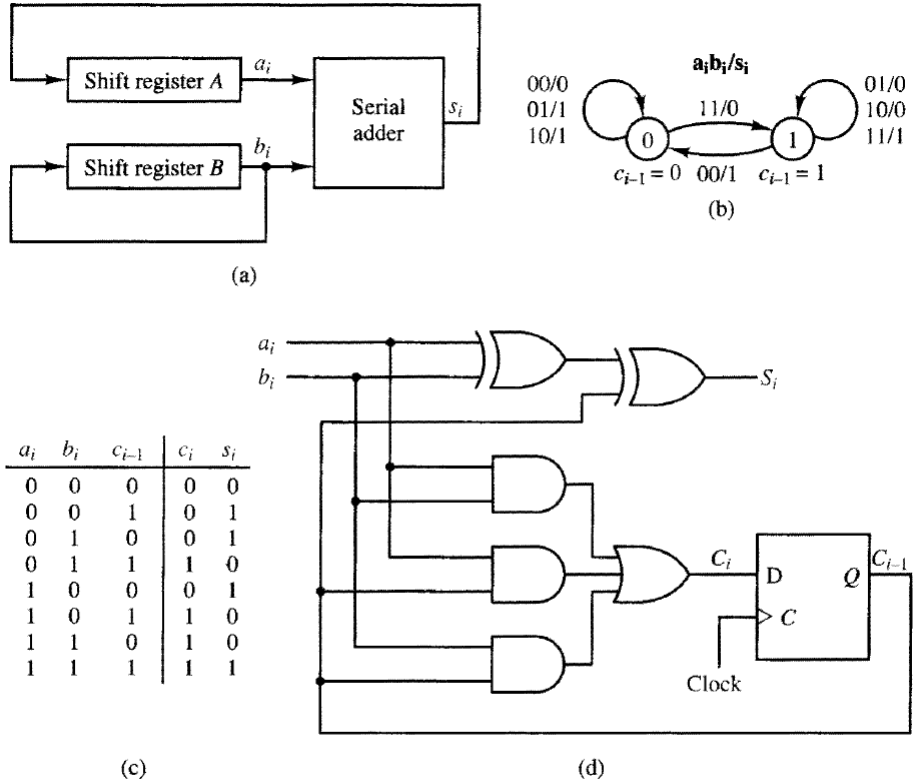
Let us design a serial binary adder that computes the sum of two n-bit binary numbers, $a_{n-1} \ldots a_1 a_0$ and $b_{n-1} \ldots b_1 b_0$, 1 bit at a time, beginning with the least significant bit.

The serial adder is to be used as illustrated in the block diagram of Fig. 8.34a. The addend and augend are stored in shift registers A and B, respectively. The sum replaces the addend in register A. Bits are presented to the serial adder from the shift registers. In clock cycle i, the adder inputs are addend bit a_i and augend bit b_i, and the output is the sum bit s_i, which is computed as the sum of the two inputs and the carry bit, c_{i-1}, produced while generating sum bit s_{i-1}. The circuit must remember the carry from the previous clock cycle. Therefore, the state of the adder in clock cycle i should reflect the value of c_{i-1}. Consequently, two states are needed for the circuit. State 0 represents the condition $c_{i-1} = 0$ and state 1 represents $c_{i-1} = 1$. At the end of each clock cycle, the sum bit s_i is shifted into register A to replace addend bit a_i, while b_i is returned to register B.

The serial adder state diagram is given in Fig. 8.34b and the corresponding state table in Fig. 8.34c. Using a D flip-flop as the memory element, it can be shown that the excitation and output equations are simply the full-adder equations, Eqs. 4.19, derived in Chapter 4.

$$s_i = a_i \oplus b_i \oplus c_{i-1}$$
$$D = c_i = a_i b_i + a_i c_{i-1} + b_i c_{i-1}$$

where c_{i-1} represents the present state and c_i the next state of the controller. The completed logic diagram is given in Fig. 8.34d.



a_i	b_i	c_{i-1}	c_i	s_i
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

Figure 8.34 Serial binary adder design. **(a)** Block diagram. **(b)** State diagram. **(c)** State table. **(d)** Logic diagram.

Finite-state Controllers

Many applications *require control circuits* that perform designated sequences of actions in response to externally applied signals or as a result of conditions produced within the circuit, such as a carry generated in an addition operation or a counter reaching a terminal value.

Such control circuits are characterized by having *a finite number of states*; hence they are referred to as finite-state controllers.

One of the most common applications of finite-state controllers is as *control units* for computers and other digital systems.

Such systems comprise two parts: a data path and a control unit.

The *data path* performs various operations on data elements, such as arithmetic operations and other transformations. Data paths typically comprise combinational logic modules, such as arithmetic logic units and multiplexers, and often include registers for data storage.

The *control unit* supplies commands to the data path to enable selected operations. These commands must be sequenced properly to ensure that the proper operations are carried out in response to various inputs and conditions.

Finite-state Controllers

We design a control unit for a digital system by identifying its *inputs* and *outputs* and then developing the *control algorithm* in the form of a state diagram.

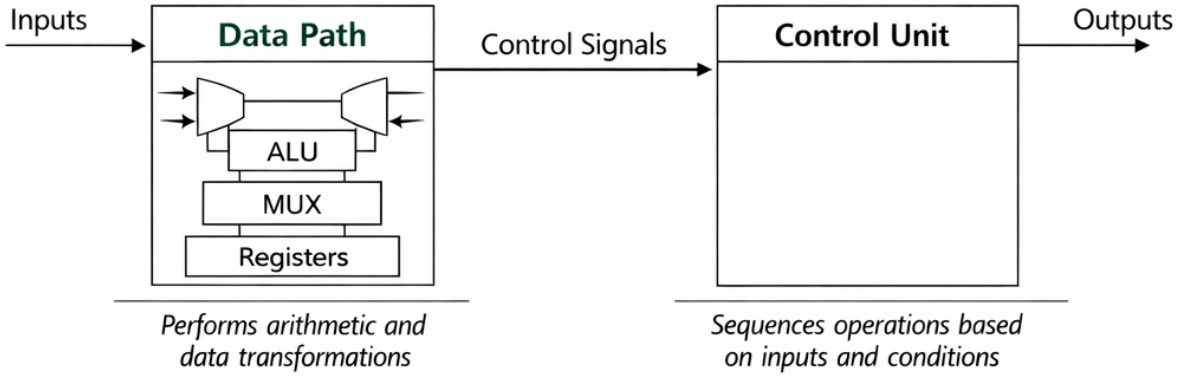
Finite-state controllers are designed using the general synchronous sequential circuit synthesis procedure defined earlier.

We begin by defining the inputs and outputs of the controller and then the algorithm to be implemented.

The *algorithm* is often specified in *state diagram* format. Then the remaining steps of the synthesis procedure are performed.

Finite-State Controllers

- Control circuits execute sequences of actions
- Triggered by **external** inputs or internal conditions
- Have a **finite** number of states → **finite-state controllers**



Identify Inputs and Outputs

⇩

Develop Control Algorithm

⇩

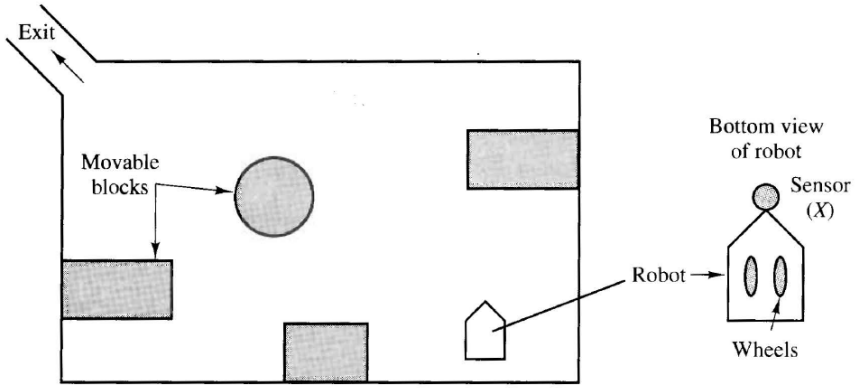
Create State Diagram

⇩

Apply Sequential Circuit Synthesis

Design Example 4 | FSC for the Robot

We wish to design a finite-state controller for the robot so that it can find its way out of the maze shown in the figure.



The robot is to maneuver by turning whenever it comes in contact with an obstacle.

On the nose of the robot is a sensor whose output $x = 1$ whenever it is in contact with an obstacle; $x = 0$ otherwise.

The robot has two control lines: $z_1 = 1$, which turns the robot to the left, and $z_2 = 1$ which turns the robot to the right.

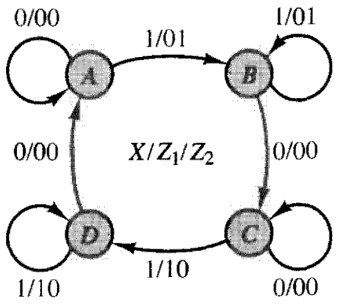
When it encounters an obstacle, the robot should turn right until no obstacle is detected.

The next time an obstacle is detected, the robot should turn left until the obstacle is cleared, and so on.

The robot controller requires four states as follows:

State A	=	no obstacle detected, last turn was left
State B	=	obstacle detected, turning right
State C	=	no obstacle detected, last turn was right
State D	=	obstacle detected, turning left

Design Example 4 | FSC for the Robot



Step 1: State Diagram

y_1y_2 \ x	0	1
A	A/00	B/01
B	C/00	B/01
C	C/00	D/10
D	A/00	D/10

NS/z_1z_2

Step 2: State Table

$A = 00$
$B = 01$
$C = 11$
$D = 10$

Step 3: State Assignment

y_1y_2 \ x	0	1
00	00/00	01/01
01	11/00	01/01
11	11/00	10/10
10	00/00	10/10

Y_1Y_2/z_1z_2

$$z_1 = xy_1$$
$$z_2 = x\bar{y}_1$$

y_1y_2 \ x	0	1
00	0	0
01	0	0
11	0	1
10	0	1

z_1

y_1y_2 \ x	0	1
00	0	1
01	0	1
11	0	0
10	0	0

z_2

Step 5: Output Table

Design Example 4 | FSC for the Robot

y_1y_2 \ x	0	1
00	00/00	01/01
01	11/00	01/01
11	11/00	10/10
10	00/00	10/10

Y_1Y_2/z_1z_2

State transitions $Q(t)$	$Q(t+\varepsilon)$	Required inputs $J(t)$	$K(t)$
0	0	0	d
0	1	1	d
1	0	d	1
1	1	d	0

y_1y_2 \ x	0	1
00	00	01
01	11	01
11	11	10
10	00	10

Y_1Y_2

y_1y_2 \ x	0	1
00	0d	0d
01	1d	0d
11	d0	d0
10	d1	d0

J_1K_1

y_1y_2 \ x	0	1
00	0d	1d
01	d0	d0
11	d0	d1
10	0d	0d

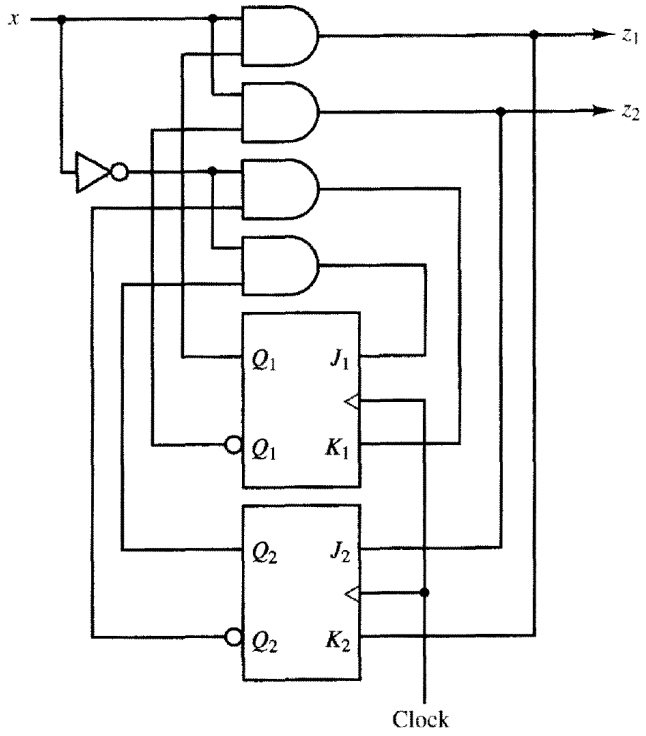
J_2K_2

Step 4: Transition Tables

$$J_1 = \bar{x}y_2$$
$$K_1 = \overline{x + y_2} = \bar{x}\bar{y}_2$$

$$J_2 = x\bar{y}_1$$
$$K_2 = \overline{\bar{x} + \bar{y}_1} = xy_1$$

Design Example 4 | FSC for the Robot



(f)

$$z_1 = xy_1$$
$$z_2 = x\bar{y}_1$$

$$J_1 = \bar{x}y_2$$
$$K_1 = \overline{x + y_2} = \bar{x}\bar{y}_2$$

$$J_2 = x\bar{y}_1$$
$$K_2 = \overline{\bar{x} + \bar{y}_1} = xy_1$$

Design Example 5 | Candy Machine

Let us design a control unit for a simple coin-operated candy machine. Candy costs 20 cents, and the machine accepts nickels and dimes. Change should be returned if more than 20 cents is deposited. No more than 25 cents can be deposited on a single purchase; therefore, the maximum change is one nickel.

The control unit has two inputs, *N* and *D,* which are outputs of the coin detector.

The coin detector generates a 1 on signal *N* if a nickel is deposited and a 1 on signal *D* output if a dime is deposited.

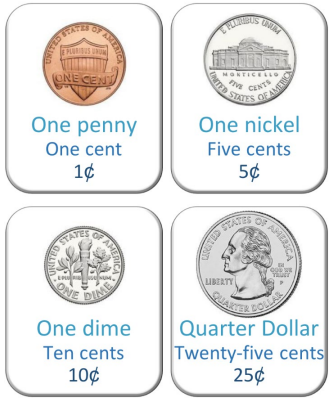
The *N* and *D* lines automatically reset to 0 on the next clock pulse.

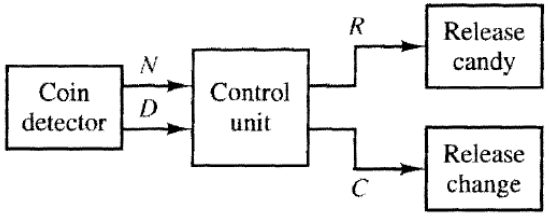
We shall assume that it is physically impossible to insert two coins at the same time, and therefore we cannot have $N = D = 1$ in the same clock period.

The control unit has two outputs, *R* and C. The candy is released by a 1 on signal *R,* and a nickel in change is released by a 1 on signal C.

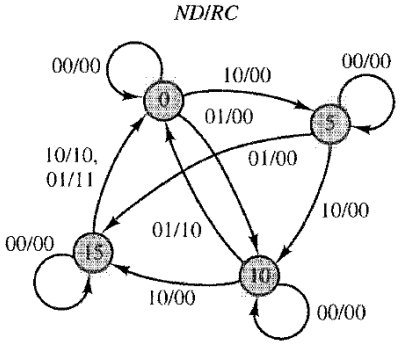
When a deposited coin increases the amount to 20 or 25 cents, the control unit will return to state 0 while releasing the candy, along with releasing change, if necessary.

Note that the input combination $ND = 11$ is not specified since two coins cannot be deposited simultaneously.



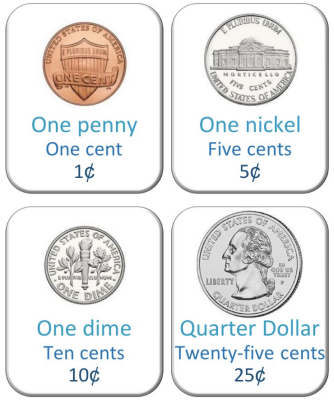


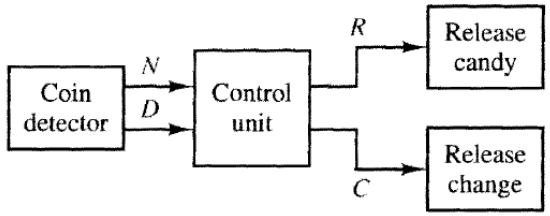
(a)



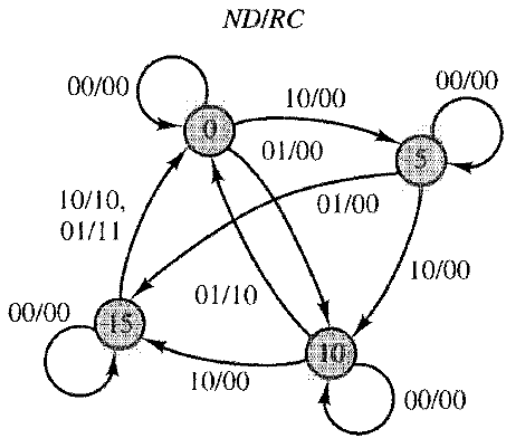
(b)

Design Example 5 | Candy Machine





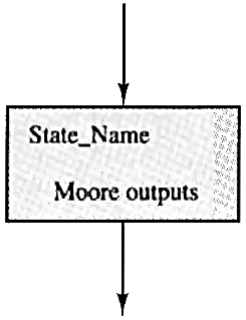
(a)

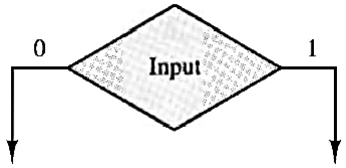


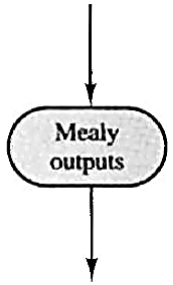
(b)

ASM | Algorithmic State Machine Diagrams

The ASM diagram is a convenient tool for expressing algorithms to be realized by sequential logic circuits, much as a flow chart might be used to describe a software algorithm to be programmed on a computer.







State box: Represents one state of the circuit, and is therefore equivalent to one node of a state diagram.

The state name is listed in the box and, for Moore circuits, the outputs to be produced, that is, the actions to be performed while in that state.

A state box always has a single entry point and a single exit point.

Decision box: Represents a state transition decision based on a test of one circuit input.

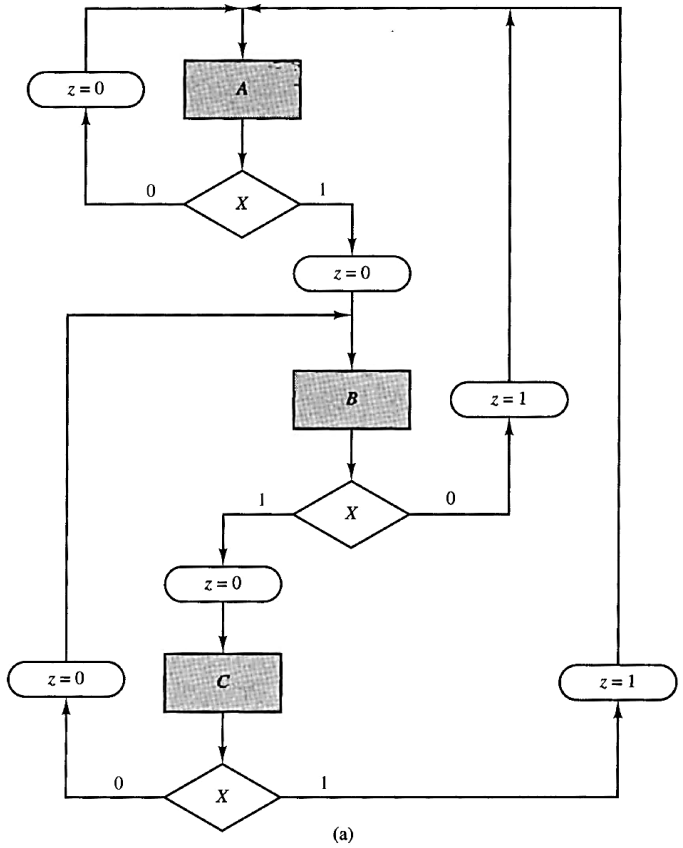
A decision box lists the input to be tested and has a single entry point and two exit points, one for the input equal to 0 and one for the input equal to 1.

A separate decision box is used for each input test.

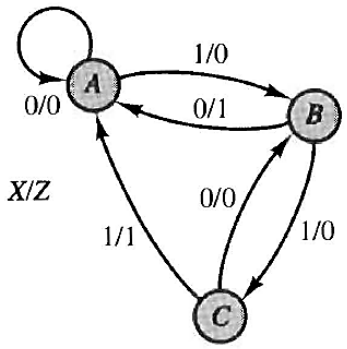
Conditional output box: Specifies outputs associated with state transitions for a given input in a Mealy circuit.

A conditional output box is placed in the path between a decision box and a state box and has a single entry and exit point.

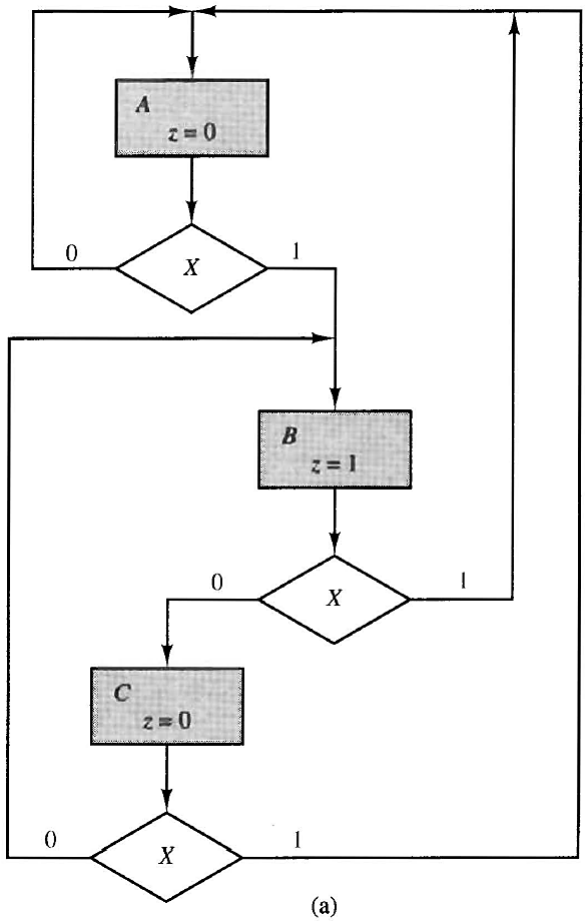
ASM Representation of a Mealy model sequential circuit.



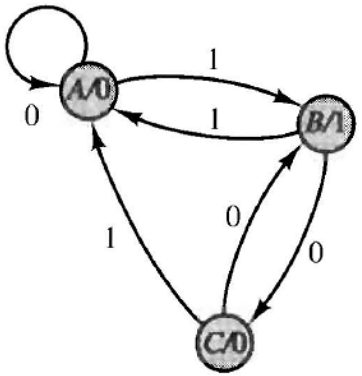
(a)



ASM Representation of a Moore model sequential circuit.



(a)



Design Example 6 | ASM

Let us design an ASM diagram for an 8-bit serial two's complementer that will implement Algorithm 1.4 presented in Chapter 1.

Algorithm 1.4 requires that we examine the bits of the number from right to left, copying the bits until the first 1 bit has been copied and then complementing the remaining bits. The ASM diagram of the serial two's complement algorithm is given in Fig. 8.45. Data bits are supplied to the circuit sequentially on input x, beginning with the least significant bit. The circuit output z is the corrected data bit. In state A, we are looking for the first 1 bit, and therefore $z = x$. In state B, the first 1 bit has been detected, and therefore $z = \bar{x}$. Note that a Mealy model has been used, since z is a function of x and the state. Therefore, all outputs are specified in conditional output boxes.

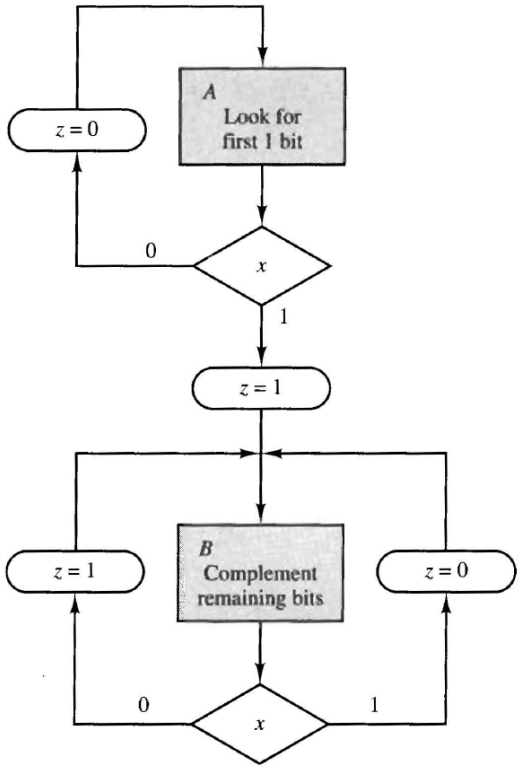


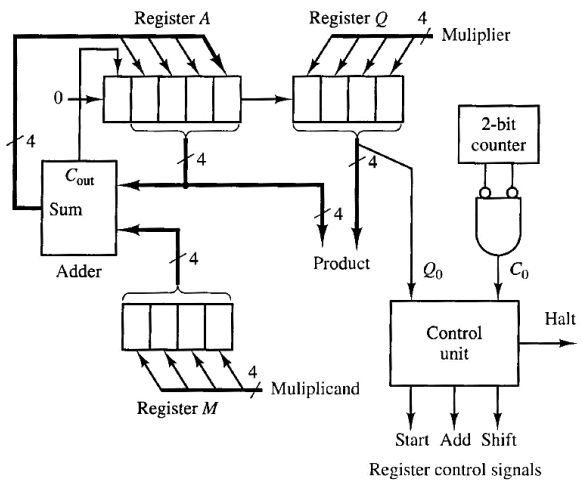
Figure 8.45 ASM representation of a serial two's complementer.

Design Example 7 | Multiplier

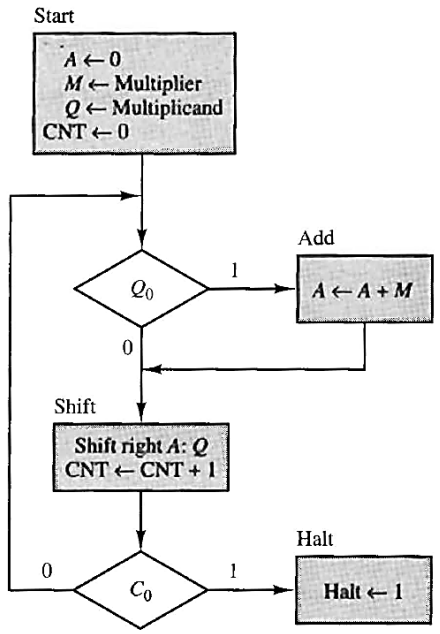
We wish to design the control unit for a binary multiplier that will compute the 8-bit product of two 4-bit unsigned binary numbers using a series of add and shift operations. The multiplier begins when pulsed by a *Reset* signal and halts with the product on its outputs. A *Halt* signal indicates the end of the operation.

We develop the multiplication algorithm by first examining the "pencil and paper" algorithm, as described in Chapter 1. Consider the product of $(0111)_2$ and $(1010)_2$.

			0	1	1	1	Multiplicand
×			1	0	1	0	Multiplier
			0	0	0	0	Partial product 1
		0	1	1	1		Partial product 2
	0	0	0	0			Partial product 3
0	1	1	1				Partial product 4
1	0	0	0	1	1	0	Product



(a)



(b)

Start: Load the operands into the M and Q registers, and clear the A register and counter *CNT*.

***Add*:** Add the multiplicand (M) to the current partial product (A) by loading the sum and carry outputs of the binary adder into the A register.

***Shift*:** Shift the partial product and multiplier 1 bit to the right by enabling the shift control lines of the A and Q registers. Also, increment the counter by enabling its increment control line.

***Halt*:** Halt, leaving the final product in the A and Q registers.

4. Incompletely Specified Circuits

Incompletely Specified Circuits

- State table contains don't-cares $(d, x, -)$.
- Only a certain set (not all) of inputs can ever be applied.
- Don't-cares may be able to be grouped with 1 in the transition table to produce a simpler circuit

Partial State Diagram

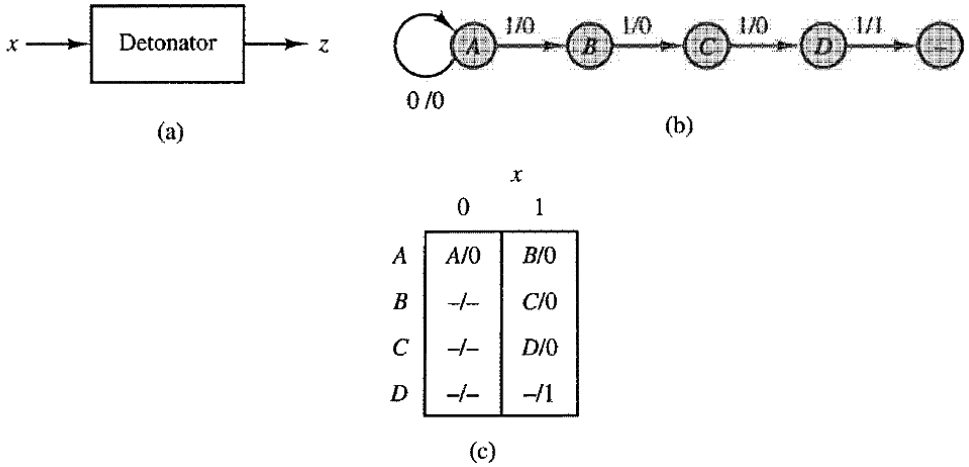


Figure 8.49 Detonator circuit. **(a)** Block diagram. **(b)** Partial state diagram. **(c)** State table.

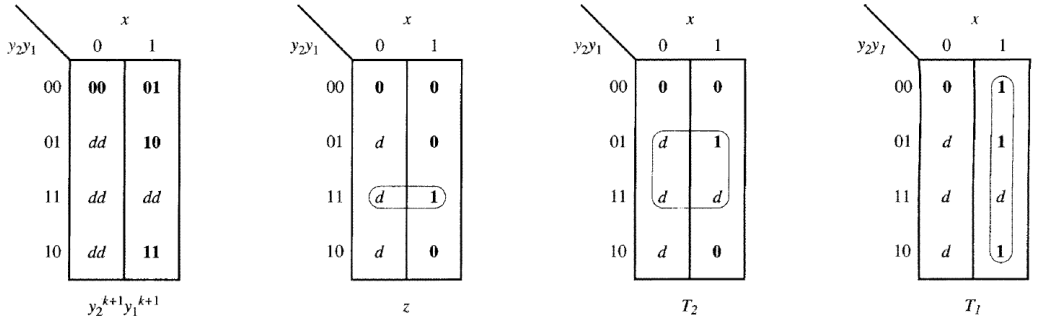


Figure 8.50 Detonator transition, output, and excitation maps.

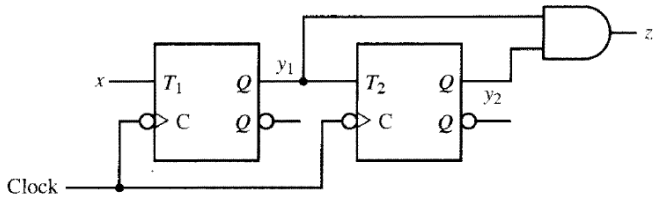


Figure 8.51 Detonator implementation.

The Realization of Incompletely Specified Circuits

For the binary state table, let us complete the realization using D, clocked T, and clocked JK flip-flops.

$$D_2 = \bar{x}y_1 + y_2y_1$$
$$D_1 = y_2\bar{y}_1$$

$$T_2 = \bar{x}y_1 + y_2\bar{y}_1$$
$$T_1 = y_2 + y_1$$

$$J_2 = \bar{x}y_1$$
$$K_2 = \bar{y}_1$$
$$J_1 = y_2$$
$$K_1 = 1$$

$$z = xy_2 + \bar{x}\bar{y}_1$$

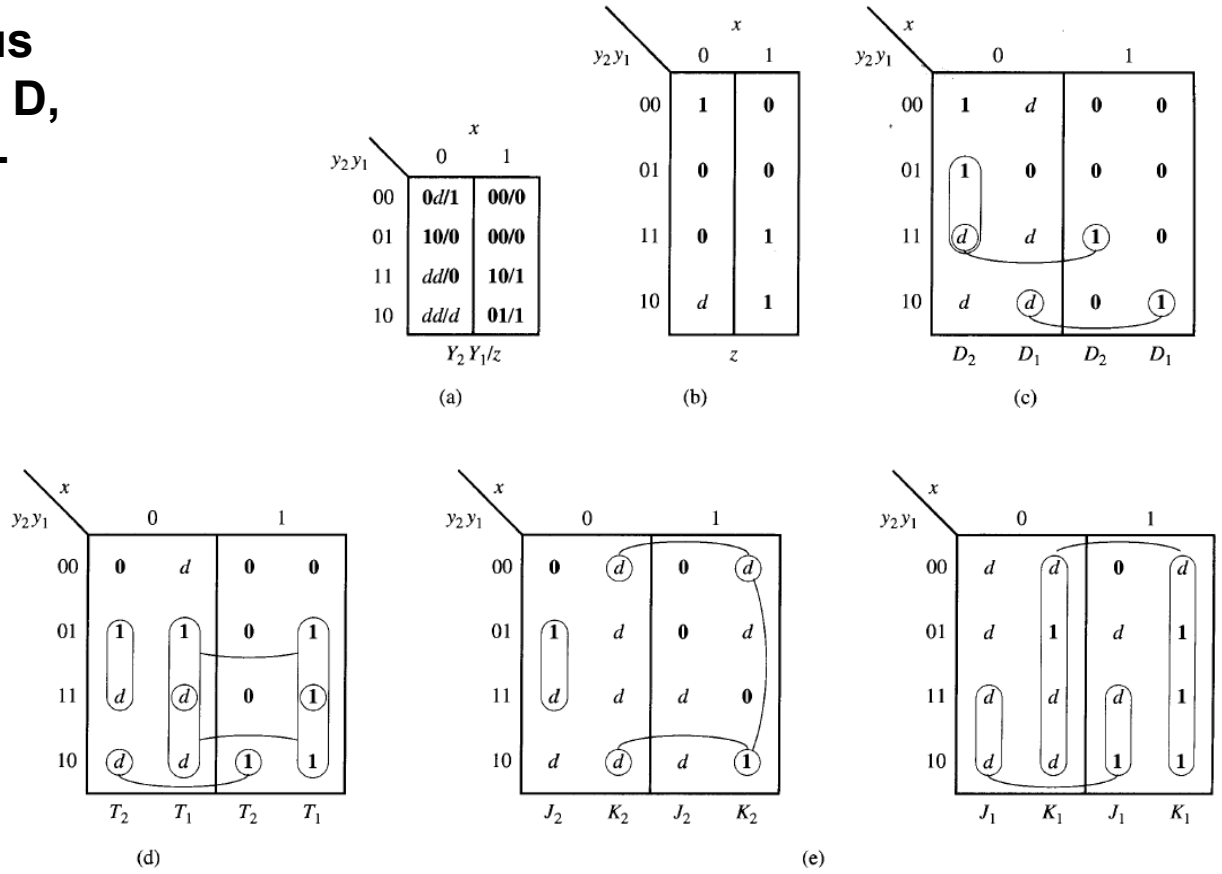


Figure 8.52 Logic realizations. **(a)** Binary state table. **(b)** K-map for z. **(c)** D flip-flops. **(d)** Clocked T flip-flops. **(e)** Clocked JK flip-flops.

THANK YOU